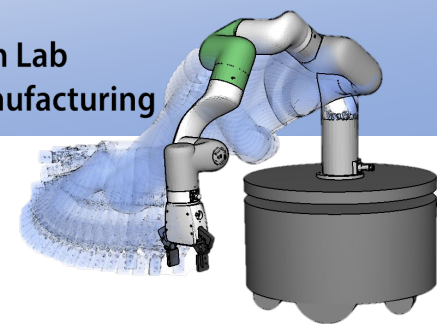


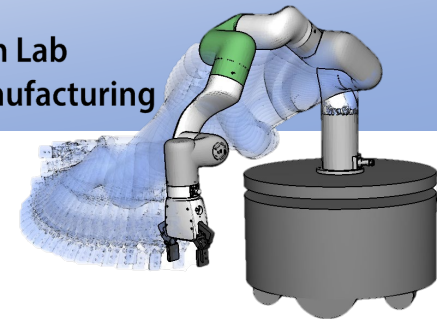


C言語：計算量・初等整 列・初等探索

原田研究室 准教授 万 偉偉

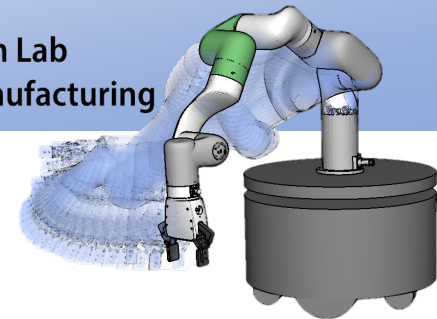
基礎工学部 F522号室

wan@sys.es.osaka-u.ac.jp



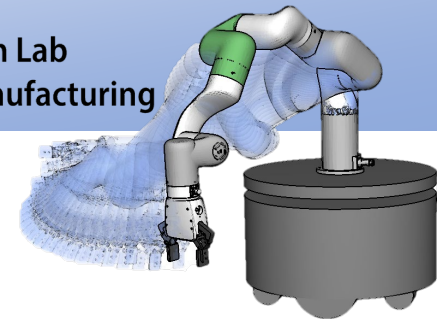
復習 pointer

- ポインタ
 - 変数の値が格納されている記憶装置の位置を指す
 - 左辺値を右辺値として持つ
- ポインタの演算
 - 代入
 - 参照
 - 前進・後進
 - 等価
- ポインタは計算機アドレスと異なり，強く型に縛られている



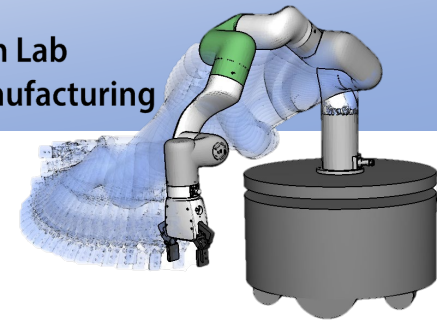
復習 string

- 文字列
 - C言語の型ではない
 - ADTとして複数の言語に実装されている
 - C言語には文字型の並びとして表現する
 - 文字列の末に終端記号 $\text{\textbackslash}0$ が必要
- C言語が提供している演算関数
 - `char *strncpy( char *dst, char *src, size_t n );`
 - `char *strncat( char *s, char *scat, size_t n );`
 - `int strcmp( char *s1, char *s2);`



復習 structure

- 宣言 : struct 構造体タグ名 変数名
- ポインタ宣言 : struct 構造体タグ名 *変数名
- メンバの参照
- 構造体の配列
- 構造体ポインタのメモリ管理
 - void *malloc(size_t size)
 - void free(void *ptr)
- 構造体宣言の簡略
 - typedef struct 構造体タグ名 変数名



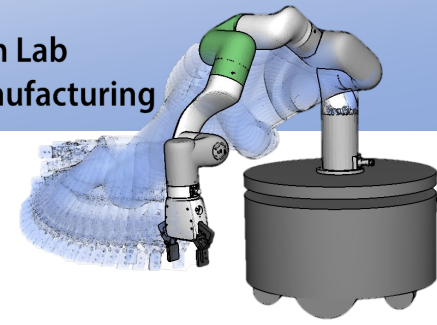
復習

- ヒープ、スタック、記憶クラス

```
25 #ifndef _FILE_DEFINED
26 struct _iobuf {
27     char *_ptr;
28     int _cnt;
29     char *_base;
30     int _flag;
31     int _file;
32     int _charbuf;
33     int _bufsiz;
34     char *_tmpfname;
35 };
36 typedef struct _iobuf FILE;
37 #define _FILE_DEFINED
38 #endif
```

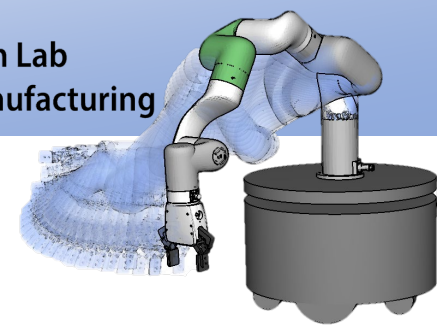
復習 file

- 文字列と同じようにC言語のない型
 - ADTとして複数の言語に実装されている
- C言語にはFILEを構造体とし定義される
- C言語が提供している操作関数
 - fopen ファイルをセットする関数
 - fclose ファイルを外す関数
 - rewind 巻き戻し（ファイル先頭へ移動）
 - fread/getc データの読み込み関数
 - fwrite/putc データの書き込み関数
 - ftell ヘッド位置を知る関数
 - fseek ヘッドを移動する関数
 - 概念：読み込みヘッド、書き込みヘッド

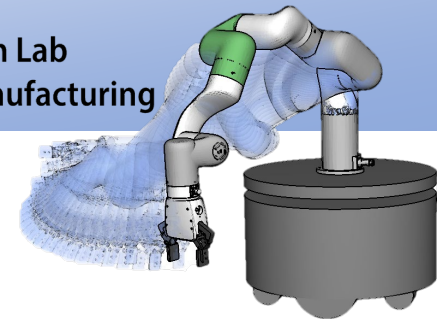


今日の内容

- 初等整列・初等ソート・計算量



計算量



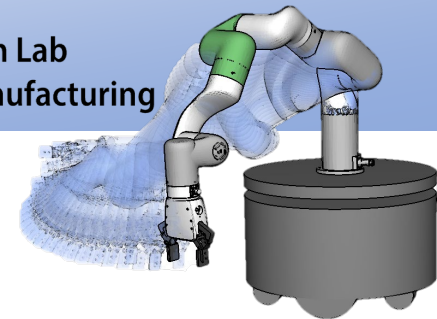
暗証ロックの解除

- x 桁の番号をもつ暗証ロック
- 2 桁であれば 00 ~ 99 の 100 個の正解
- 0 番から順に入力→最悪 100 手目を開く

- * 0 であるときの確率は $1/100$ で、このときの手数は 1 手
- * 1 であるときの確率は $1/100$ で、このときの手数は 2 手
- * 2 であるときの確率は $1/100$ で、このときの手数は 3 手
- ⋮
- * 99 であるときの確率は $1/100$ で、このときの手数は 100 手

平均手数は

$$1/100 \times 1 + 1/100 \times 2 + 1/100 \times 3 + \dots + 1/100 \times 100 = 50.5$$



計算量の実際

- 問題の大きさを d ($s=10^d$) とする

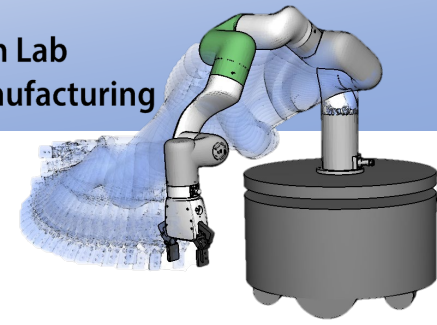
問題の大きさ	$d = 1$	$d = 2$	$d = 5$	$d = 10$	$d = 20$
最悪の手数	10 手	100 手	10^5 手	10^{10} 手	10^{20} 手
手入力の時間	10 秒	100 秒	27.8 時間	316.9 年	3.16 兆年

- Core i7 プロセッサーで解く

- 一秒間の計算回数：159億回（15,900MIPS）

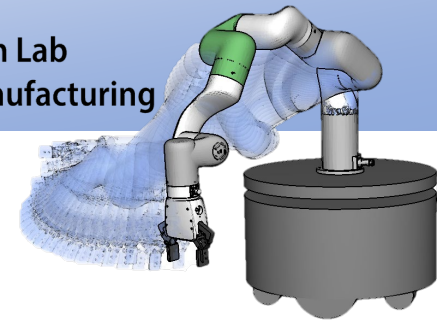
問題の大きさ	$d = 1$	$d = 2$	$d = 5$	$d = 10$	$d = 20$
最悪の手数	10 手	100 手	10^5 手	10^{10} 手	10^{20} 手
Core i7	0.63E-09	0.63E-08	0.63E-05	0.63	0.63E+10

200年



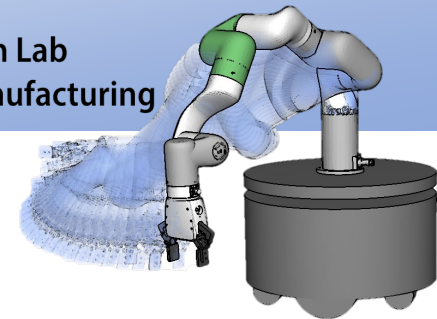
イメージ的な計算量の説明

- 各データを1回処理する場合
 - $O(n)$
- 各データを全組み合わせで処理する場合
 - $O(n^2)$
- 分岐によりデータを探索する場合
 - $O(\log n)$



正しい計算量の定義

- 実行時間がデータの大きさ n に比例
 - $O(n)$
- 実行時間がデータの大きさの2乗に比例
 - $O(n^2)$
- 実行時間がデータの大きさの $\log$ に比例
 - $O(\log n)$



計算量の表し方

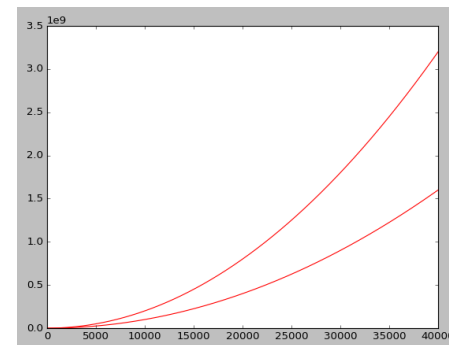
- 実際の処理が, $2*n^2+5n+1000$ の場合



最大次数以外は無視
最大次数の係数は無視

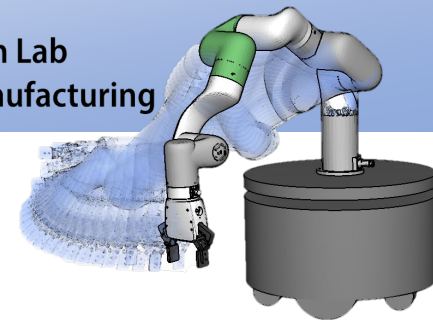


$O(n^2)$

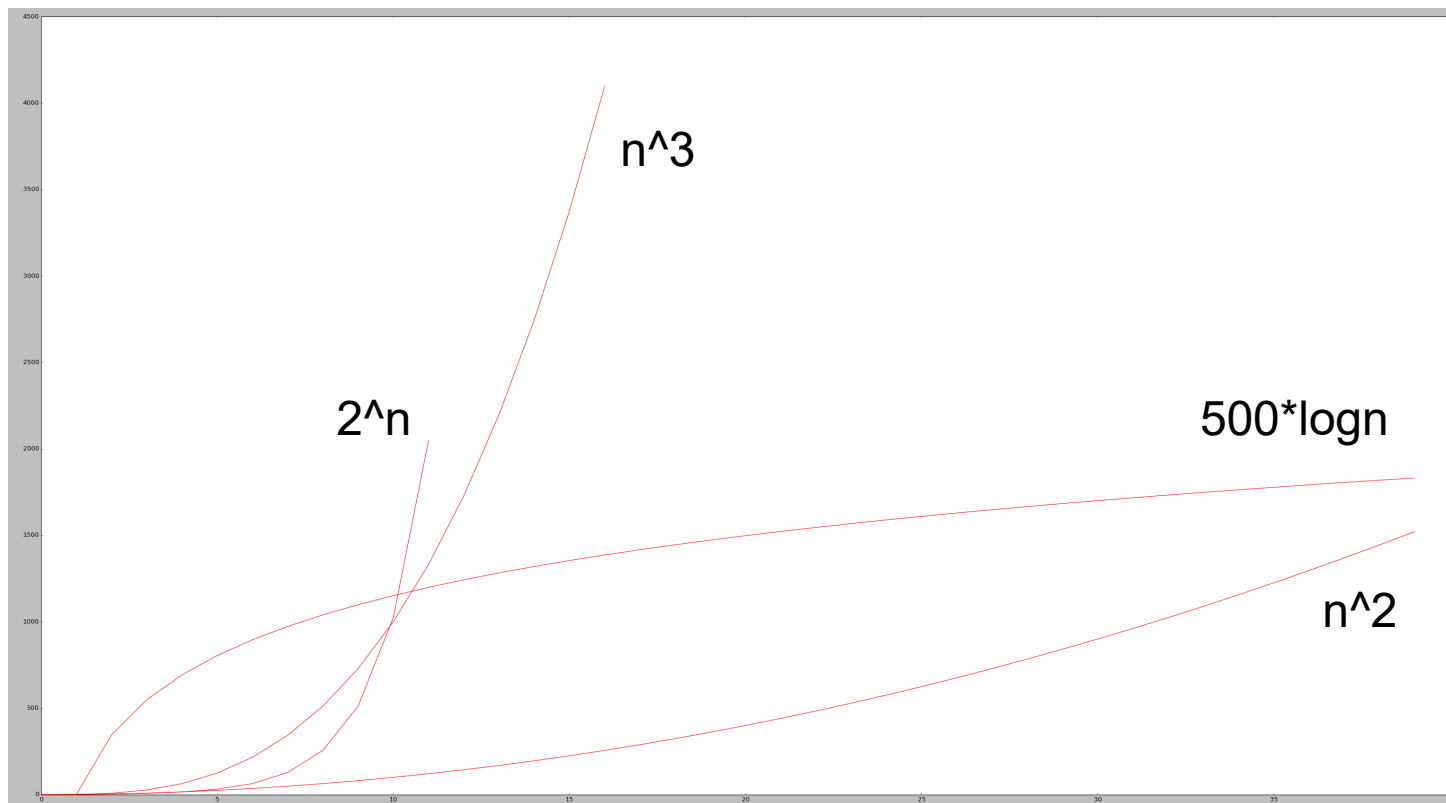


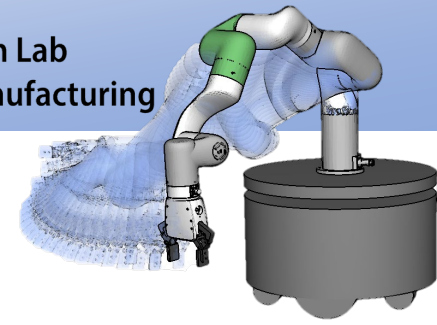
n が1000の場合, $2*n^2+5n+1000 = 2006000$
 $n^2 = 1000000$

n の増加幅
に対して,
それほど値は
変わらない



各計算量の曲線

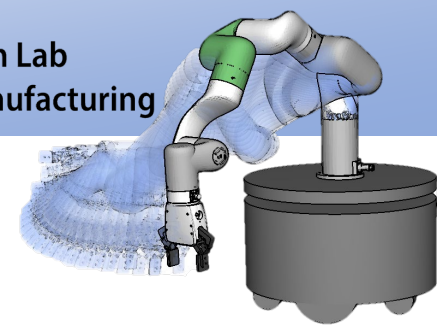




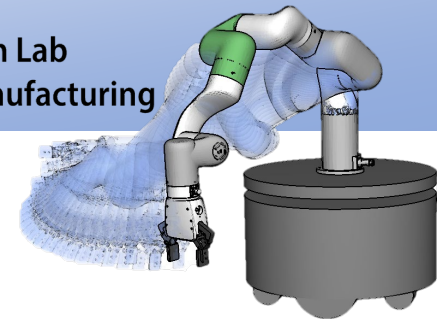
計算量の計算

```
1. i=1;  
2. while(i<=n)  
3.     i=i*2;
```

```
1. sum=0 ;  
2. for(i=1;i<=n;i++)  
3.     for(j=1;j<=n;j++)  
4.         sum++ ;
```

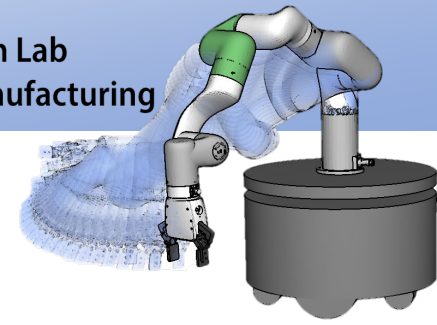


初等の整列



整列(sort)

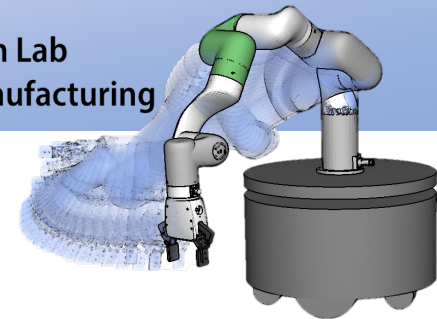
- ある基準(<) でデータを並べ替えること
 - 基準(<)は各自が定義する.
 - 数値の場合はすでに大小関係が定義されている



整列(sort)

- 内部整列(internal sort)
 - 主記憶上で行う整列
- 外部整列(external sort)
 - 外部記憶装置（HD等）上で行う

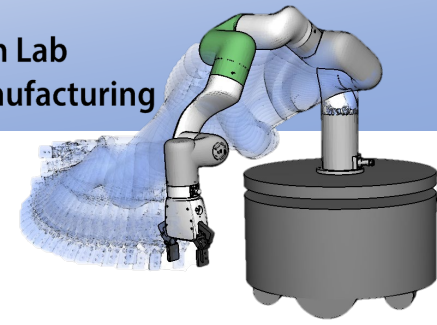
整列アルゴリズムの優劣→比較回数と交換回数の大小



安定な整列(stable sort)

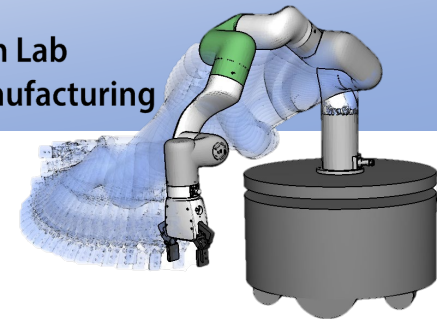
- 同じ値を持つデータ間の順序関係が整列の前後で保たれている

例) 整列前 : 7 9 5_A 4 8 5_B 2
整列後 : 2 4 5_A 5_B 7 8 9 (安定)
整列後 : 2 4 5_B 5_A 7 8 9 (不安定)



安定な整列(stable sort)

- 選択ソート
 - 未整列部分から最小要素を発見し， 整列部分の後の要素とスワップ.
- バブルソート
 - 後から近接要素のみスワップ. これの2回繰り返し.
- 挿入ソート
 - 未整列部分の先頭要素を， 整列部分の適切な位置に挿入



単純な整列（選択ソート）

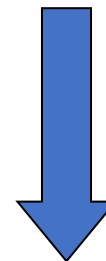
- 未整列部分から最小の要素を選び出し，それを未整列部分の先頭と入れ替える

$$a[0] \leq a[1] \leq \dots \leq a[i-1] \leq a[i] \leq \dots a[n-1]$$

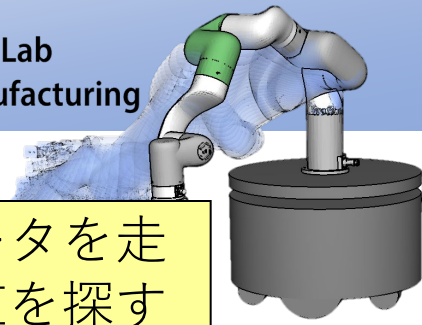
整列済

未整列部分

未整列部分から最小値を取り出し
続けると下記の条件を満たす
整列アルゴリズムが満たす事後条件



$$\forall i, j [i < j] \Rightarrow a[i] \leq a[j]$$



選択ソート

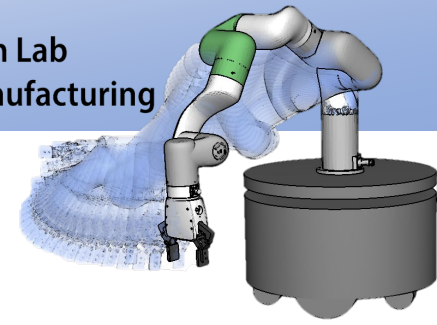
一度全てのデータを走査して，最小値を探す

整列前
1回目

20 6 55 74 3 45 13 87 46 30

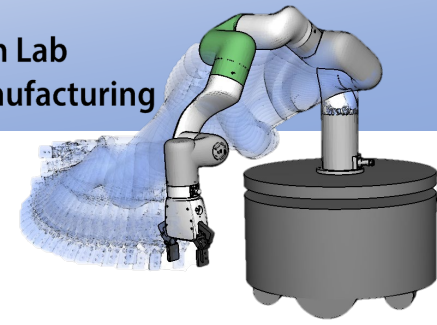
入替





選択ソート

整列前	20	6	55	74	3	45	13	87	46	30
1回目	3	6	55	74	20	45	13	87	46	30

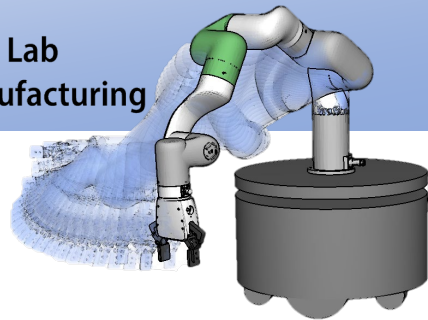


選択ソート

整列前	20	6	55	74	3	45	13	87	46	30
1回目	3	6	55	74	20	45	13	87	46	30
2回目	3	6	55	74	20	45	13	87	46	30

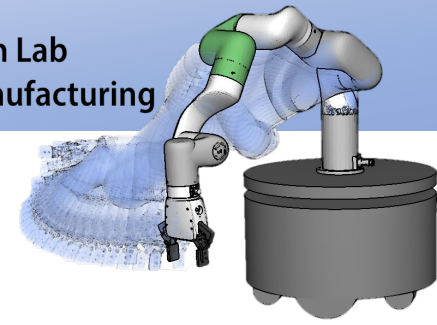


入替



選択ソート

整列前	20	6	55	74	3	45	13	87	46	30
1回目	3	6	55	74	20	45	13	87	46	30
2回目	3	6	55	74	20	45	13	87	46	30
3回目	3	6	13	74	20	45	55	87	46	30
4回目	3	6	13	20	74	45	55	87	46	30
5回目	3	6	13	20	30	45	55	87	46	74
6回目	3	6	13	20	30	45	55	87	46	74
7回目	3	6	13	20	30	45	46	87	55	74
8回目	3	6	13	20	30	45	46	55	87	74
9回目	3	6	13	20	30	45	46	55	74	87

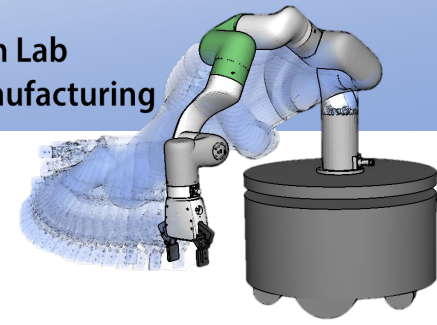


選択ソート（疑似コード）

```
for (i ← 0..n-2)
  lowest ← argmini < j < n (a[j])
  swap(a[i], a[lowest])
```

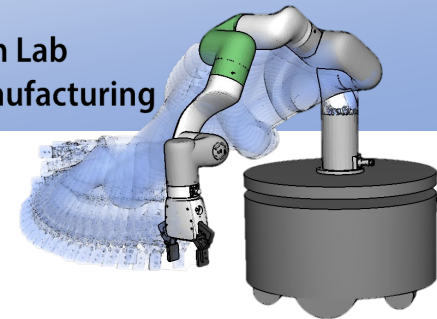


配列 $a[]$ の i 以降の要素
で最小値の要素番号を求める



選択ソートの計算量

- ループ n 回, argmin の計算量 $O(n)$
- 選択ソートの計算量は $O(n^2)$

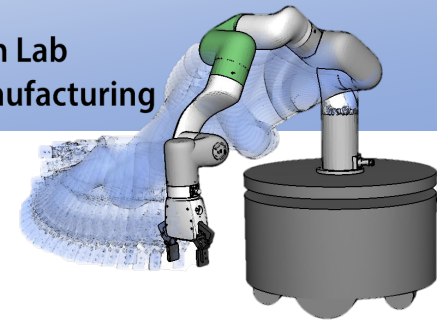


単純な整列（バブルソート）

- 配列の後ろから先頭に向かって走査し，もし隣り合う二つの要素の大小関係が逆であったら入れ替える

整列前 20 6 55 74 3 45 13 87 46 30
1回目

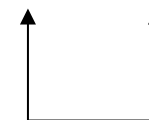




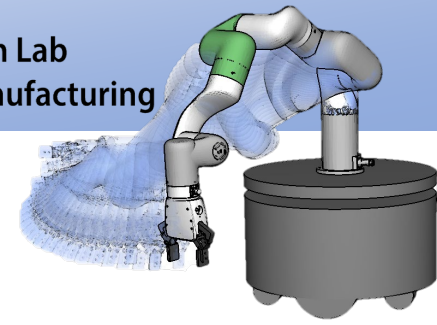
単純な整列（バブルソート）

- 配列の後ろから先頭に向かって走査し，もし隣り合う二つの要素の大小関係が逆であったら入れ替える

1回目 20 6 55 74 3 45 13 87 30 46



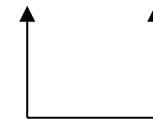
入替



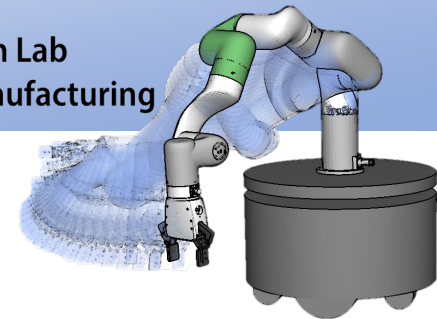
単純な整列（バブルソート）

- 配列の後ろから先頭に向かって走査し，もし隣り合う二つの要素の大小関係が逆であったら入れ替える

1回目 20 6 55 74 3 45 13 87 30 46



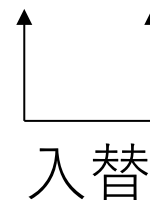
比較

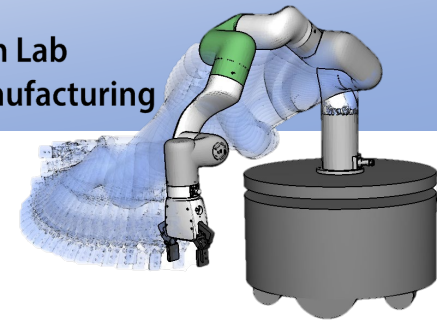


単純な整列（バブルソート）

- 配列の後ろから先頭に向かって走査し，もし隣り合う二つの要素の大小関係が逆であったら入れ替える

1 回目 20 6 55 74 3 45 13 30 87 46



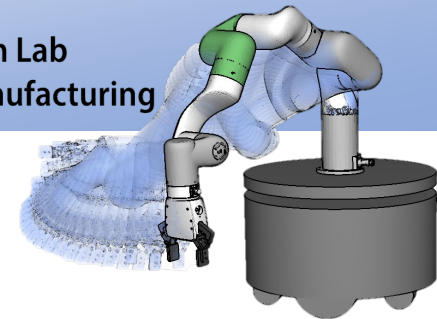


単純な整列（バブルソート）

- 配列の後ろから先頭に向かって走査し，もし隣り合う二つの要素の大小関係が逆であったら入れ替える

1回目 20 6 55 74 3 45 13 30 87 46

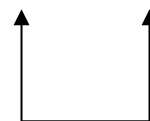




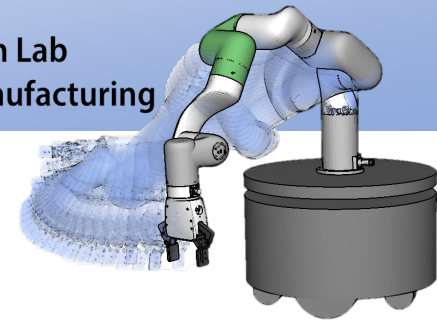
単純な整列（バブルソート）

- 配列の後ろから先頭に向かって走査し，もし隣り合う二つの要素の大小関係が逆であったら入れ替える

1 回目 20 6 55 74 3 45 13 30 87 46



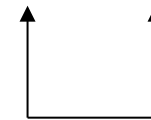
比較



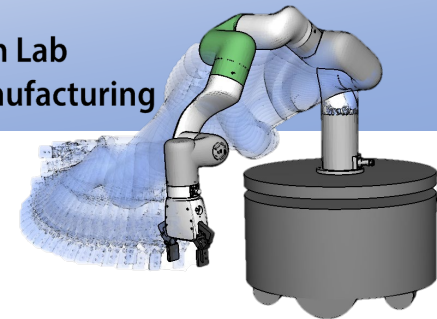
単純な整列（バブルソート）

- 配列の後ろから先頭に向かって走査し，もし隣り合う二つの要素の大小関係が逆であったら入れ替える

1 回目 20 6 55 74 3 13 45 30 87 46



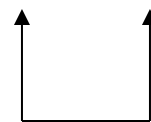
入替



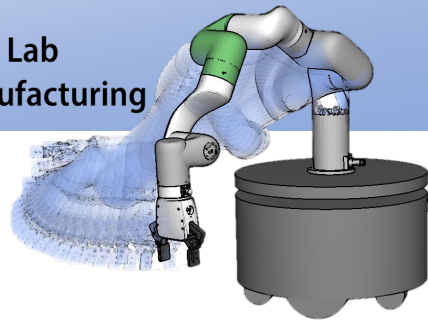
単純な整列（バブルソート）

- 配列の後ろから先頭に向かって走査し，もし隣り合う二つの要素の大小関係が逆であったら入れ替える

1回目 20 6 55 74 3 13 45 30 87 46



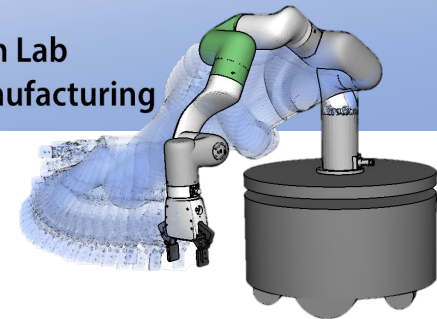
比較



単純な整列（バブルソート）

- 配列の後ろから先頭に向かって走査し，もし隣り合う二つの要素の大小関係が逆であったら入れ替える

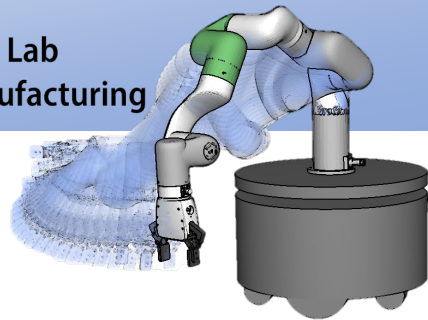
1回目	20	6	55	74	3	13	45	30	87	46
	20	6	55	3	74	13	45	30	87	46
	20	6	3	55	74	13	45	30	87	46
	20	3	6	55	74	13	45	30	87	46
	3	20	6	55	74	13	45	30	87	46



単純な整列（バブルソート）

- 配列の後ろから先頭に向かって走査し、もし隣り合う二つの要素の大小関係が逆であったら入れ替える

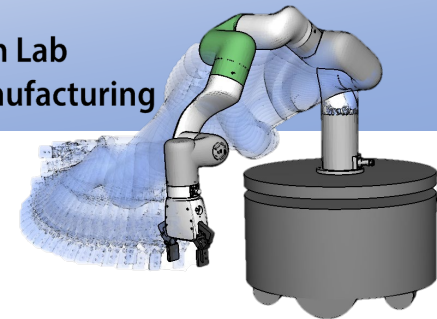
2 回目	3	20	6	55	74	13	45	30	87	46
									46	87
								30	46	
							30	45		
						13	30			
					13	74				
				13	55					
			6	13						
		6	20							



単純な整列（バブルソート）

- 配列の後ろから先頭に向かって走査し，もし隣り合う二つの要素の大小関係が逆であったら入れ替える

2回目	3	6	20	13	55	74	30	45	46	87
3回目	3	6	13	20	30	55	74	45	46	87
4回目	3	6	13	20	30	45	55	74	46	87
5回目	3	6	13	20	30	45	46	55	74	87
6回目	3	6	13	20	30	45	46	55	74	87
7回目	3	6	13	20	30	45	46	55	74	87
8回目	3	6	13	20	30	45	46	55	74	87
9回目	3	6	13	20	30	45	46	55	74	87



バブルソート（疑似コード）

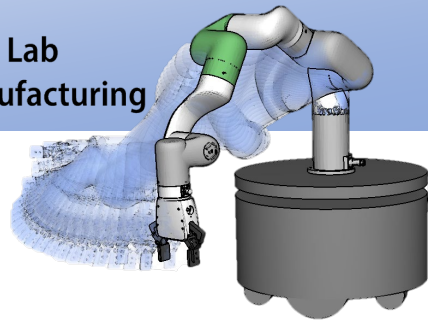
```
for (i ← 0..n-1)
  for (j ← n-1..i)
    if !(a[j-1] ≤ a[j]) swap(a[j-1], a[j])
```

右辺値の交換

整列アルゴリズムが満たす事後条件

$$\forall i, j [i < j] \Rightarrow a[i] \leq a[j]$$

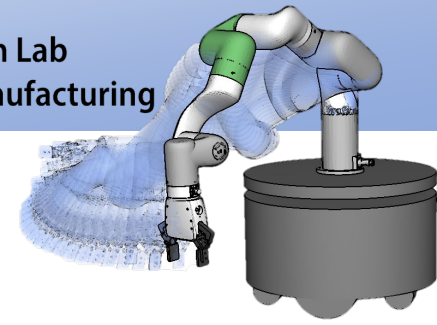
計算量は $O(n^2)$



単純な整列（挿入ソート）

- 配列の一部分が整列済みの時に，残りの要素を一つずつ整列済みの中に挿入する

整列前	20	6	55	74	3	45	13	87	46	30
1回目	6	20	55	74	3	45	13	87	46	30
2回目	6	20	55	74	3	45	13	87	46	30
3回目	6	20	55	74	3	45	13	87	46	30
4回目	3	6	20	55	74	45	13	87	46	30
5回目	3	6	20	45	55	74	13	87	46	30
6回目	3	6	13	20	45	55	74	87	46	30
7回目	3	6	13	20	45	55	74	87	46	30
8回目	3	6	13	20	45	46	55	74	87	30
9回目	3	6	13	20	30	45	46	55	74	87



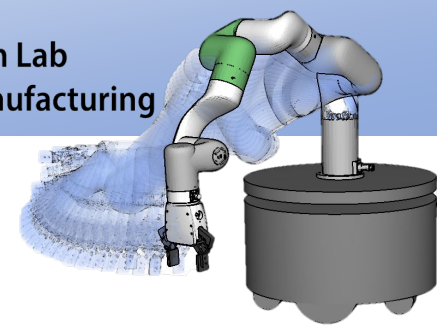
挿入ソート（疑似コード）

```
for (i ← 1..n)
  j ← i
  while (j >= 1 && ! (a[j-1] ≤ a[j]))
    swap( a[j-1], a[j] )
    j ← j-1
  endwhile
endfor
```

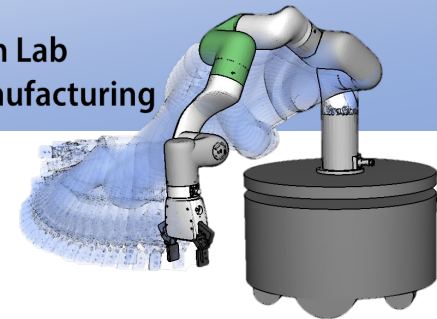
計算量：外側ループ $O(n)$

内側ループ $O(n)$

合計： $O(n^2)$

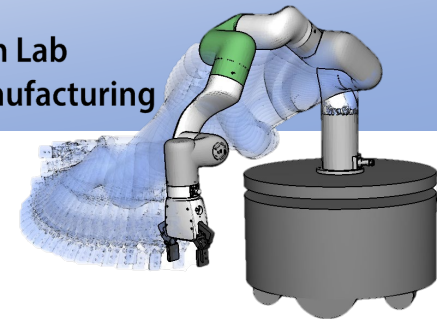


初等の探索



線形探索と二分探索

- 線形探索法(linear search)
 - データを最初から順番に探索する
 - 例) {2, 4, 5, 8, 9, 11, 6, 7, 15, 20} から15を探す
最初の要素から始めて9回目→計算量 $O(n)$
- 二分探索法(binary search)
 - データをあらかじめソートしておく, 中央の要素から検索する.

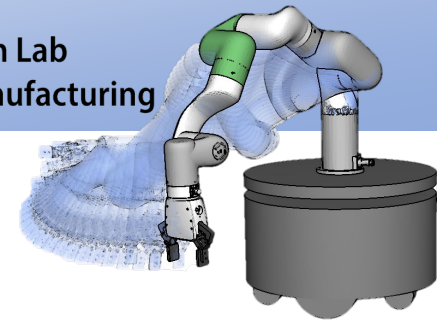


二分探索(binary search)

- $\{2, 4, 5, 6, 7, 8, 9, 11, 15, 20\}$ から15を探す

1 回目

{2, 4, 5, 6, 7, 8, 9, 11, 15, 20}



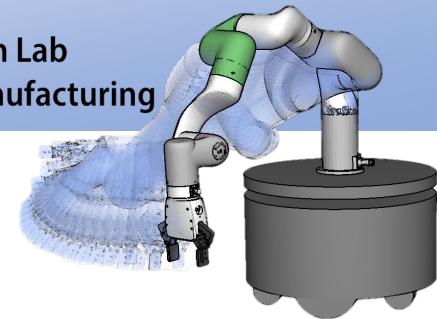
二分探索(binary search)

- $\{2, 4, 5, 6, 7, 8, 9, 11, 15, 20\}$ から15を探す

2 回目

~~{2, 4, 5, 6, 7, 8, 9, 11, 15, 20}~~

~~2, 4, 5, 6, 7, 8, 9, 11, 15, 20~~



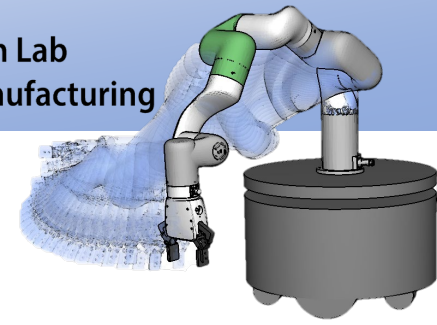
二分探索(binary search)

- $\{2, 4, 5, 6, 7, 8, 9, 11, 15, 20\}$ から15を探す

3 回目

~~{2, 4, 5, 6, 7, 8, 9, 11, 15, 20}~~

探索回数	3 回
計算量	$\log_2(n)$ $O(\log n)$



まとめ

- 計算量
- 初等整列
- 初等ソート
 - バブルソート
 - 選択ソート
 - 挿入ソート
- 初等探索